

1 Why Transparency / Interpretability?

Interpretability = turning model weights & activations into **concepts humans can understand**. AI systems are black boxes; we want to know **why they do what they do**.

Motivation: transparency tools give clarity about a model's inner workings; small model changes can shift internal representations a lot; transparency helps **monitors detect deception & other hazards** and detect **worst-case misalignments**.

Q5 & Q10: Top-down interpretability = **locates information without fully understanding how it is processed**; it looks at representational spaces while mechanistic reverse-engineers at the neuron level.

Two paradigms (Q10):

- **Mechanistic (bottom-up)** — reverse-engineer neural nets at the **neuron / node-to-node** level; explain neurons & connected circuits (excitatory vs inhibitory)
- **Top-down** — **locate information in a model without fully understanding how it is processed**; looks at **representational spaces**; far more tractable than fully reverse-engineering large models

2 Pixel Attribution (Saliency) Methods

Pixel attribution = highlight the pixels relevant to a classification (Q1). Many names: sensitivity map, saliency map, feature attribution/relevance. It's a special case of **feature attribution** (features = pixels, tabular values, or words).

Two families:

- **Occlusion / perturbation-based** — **manipulate parts of the input** to see the effect; model-agnostic (e.g. SHAP, LIME) (Q2)
- **Gradient-based** — compute the gradient of the class score w.r.t. input pixels; methods differ in how the gradient is computed. $E_{\text{grad}}(x) = \partial S_i / \partial x$

Specific gradient methods:

- **Vanilla gradient / Saliency map** — forward pass, gradient of class score w.r.t. input, visualise magnitude
- **SmoothGrad** — add **noise** to many copies of the image, average the attribution maps (reduces noise)
- **Guided Backprop** — backprop with negative activations & gradients **zeroed out** (only positive gradients)
- **Grad-CAM** — gradient-weighted class activation mapping over conv layers
- **Optimized masks** — learn a mask that blurs salient regions (very sensitive to hyperparameters)

3 Saliency Maps Can Be Deceptive — Sanity Checks

Many transparency tools make **pretty visualizations that don't actually inform** us about the model. Attention maps can 'explain' the same husky image as both a husky and a flute.

Sanity check (cascading randomization): randomize the model's layers top-to-bottom. If a saliency map **barely changes** when layers are randomized, it is **not capturing what the model learned**. A good map should **change significantly** as the model is randomized (Q3).

Q3: if a model genuinely captures higher-level class concepts, its saliency map **should change significantly** when its layers are randomized.

4 LIME & SHAP (model-agnostic explanations)

LIME = Local Interpretable Model-agnostic Explanations. Works on any **black-box** model & many data types; internals hidden. Fits a **simple interpretable model locally** around one prediction. Explanations are **locally faithful but not necessarily globally**.

LIME objective (Q8):

- $\xi(x) = \operatorname{argmin}_{g \in G} L(f, g, \pi_x) + \Omega(g)$
- $L(f, g, \pi_x)$ = how **unfaithful** the explanation g is in approximating f **locally** (weighted by proximity π_x)
 - $\Omega(g)$ = **complexity / interpretability penalty** of g (keep it simple)

From the paper "Why Should I Trust You?" (Ribeiro et al.).

SHAP = SHapley Additive exPlanations. A **game-theoretic** approach: uses **Shapley values** (Q11) from cooperative game theory to fairly allocate each feature's contribution to a prediction relative to a base rate.

Pros & cons of gradient-based vs LIME/SHAP:

- Gradient methods: visual, easy, **faster & cheaper** than model-agnostic LIME/SHAP
- But: hard to know if an explanation is **correct**; **very fragile** — adversarial perturbations give the same prediction but different maps

Q12 (select all) — true of LIME & SHAP: both are **model-agnostic (work on any black box)** AND both are **computationally expensive vs direct gradient methods**. (They are NOT global-only, and NOT restricted to images.)

5 Feature Visualization & ProtoPNet

Feature visualization — to see what an internal component detects, **synthesize an image via gradient descent that maximally activates** that neuron/channel (Neuron viz, Channel viz, or find maximally-activating natural images). See OpenAI Microscope.

ProtoPNet ('This Looks Like That') — classifies based on the most important **patches of training images** that are **prototypical of the class**; the prediction is explained by 'this region looks like that prototype'.

6 Probing

Probing = methods to **check whether a model's representations reliably encode information for a specific task** (Q6). I.e. is there a **reliable signal** in the representations to do task X?

Why: a model can score high on a task **without** actually doing the sub-task we care about; neural nets learn to 'delete' information — probing checks if the model holds onto distinctions we care about (e.g. 'Probing Negation in Language Models').

Q6: primary function of probing = **check whether representations encode information reliably** (NOT injecting trojans, NOT latent-space interpolation, NOT full reverse-engineering).

7 Trojan Attacks (backdoors) & Data Poisoning

Trojan / backdoor: adversaries implant **hidden functionality**; when a secret **trigger** appears the model has a sudden dangerous behaviour change (e.g. a sticker makes a Stop sign misread). Named after the Trojan Horse.

Attack vectors (Q7): how adversaries implant functionality — **public datasets** (poison text/image), **model-sharing libraries** (HuggingFace / PyTorch Hub / TensorFlow Hub), and a **downloadable execution file**. (API keys are NOT an attack vector here.)

Data poisoning: poison the dataset so the model gets a Trojan. Works even when a **tiny fraction (e.g. 0.05%)** of data is poisoned; triggers are hard to spot manually. In RL, perturbing observations can induce a chosen action at a target state.

Q7 (select all) — Attack Vectors: Public Datasets, HuggingFace models, a downloadable execution file (NOT API keys).

8 Detecting & Removing Trojans

Detection — two questions: (a) does a given **input** have a trigger? (b) does a given **network** have a Trojan? Hard because nets are high-dimensional (which first-layer MNIST weights are Trojaned?).

Neural Cleanse: if we know the general attack form, **reverse-engineer the trigger** by searching for a (mask m , pattern Δ) and target label; repeat for every possible target and find the **minimum** Δ needed to misclassify. The Trojan class needs a suspiciously **small** trigger vs others (anomaly index).

Neural Cleanse (Q9): its **primary goal is to reverse-engineer potential triggers** — it may not recover the exact original trigger, but reliably indicates **whether a network has a Trojan**.

Removing Trojans: the reverse-engineered trigger activates similar internal features as the original, so **pruning the affected neurons removes the Trojan**.

Meta-Networks: train a network to analyse other networks — feed clean & Trojaned nets, train a classifier on their outputs (but building such a dataset is expensive).

Q9: primary goal of Neural Cleanse = to reverse-engineer potential triggers.

9 Outlook

Powerful **detectors for hidden functionality** would make AI systems much safer; **removing hidden functionality can increase alignment** and make models less inherently hazardous. Interpretability + probing + Trojan detection all feed into monitoring for deception and misalignment.